

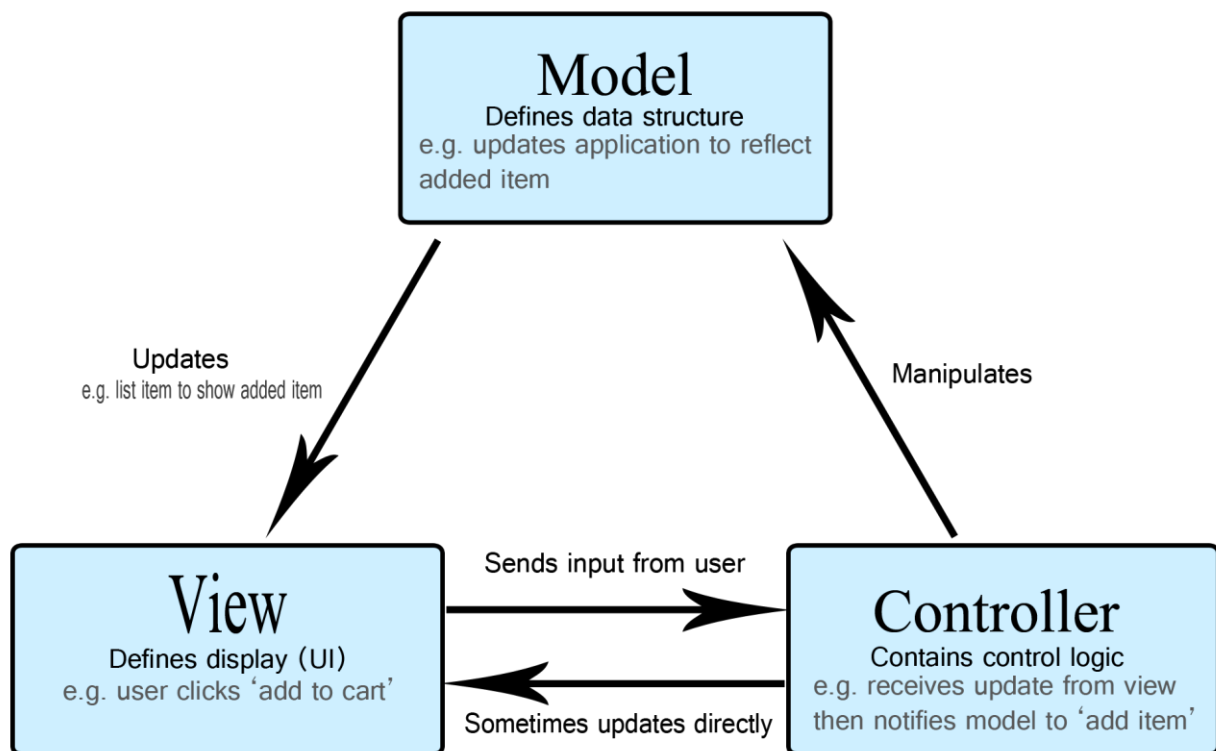
CA-Software

Assignment-1

MVC(MODEL-VIEW-CONTROLLER)

What is MVC?

The **MVC (Model-View-Controller)** design pattern is used to separate the concerns of an application into three main components: the **Model**, which handles data and business logic; the **View**, which manages the user interface; and the **Controller**, which processes user input and updates the Model. This separation makes the application more organized, easier to test, and simpler to maintain.



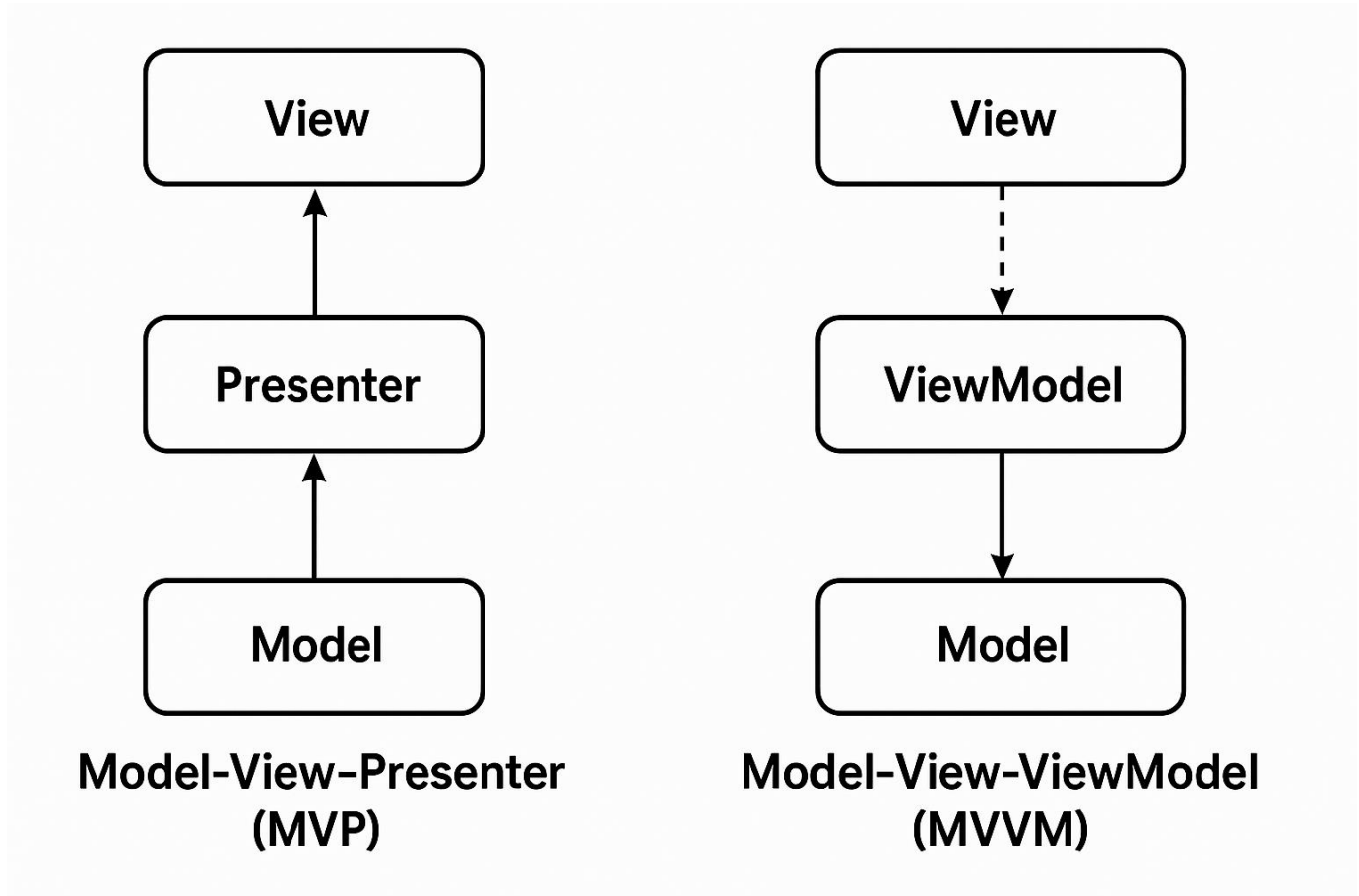
Variants of MVC

Variant 1: Model-View-Presenter (MVP)

In the MVP pattern, the **View** is responsible for displaying the UI and forwarding user actions to the **Presenter**. The **Presenter** acts as the middleman — it handles all logic, communicates with the **Model** for data, and then updates the **View** directly. The View is kept simple and passive, making it easier to test and reuse. This pattern is commonly used in **Android applications** and **Windows Forms**.

Variant 2: Model-View-ViewModel (MVVM)

In the MVVM pattern, the **View** is bound directly to the **ViewModel**, which holds the presentation logic and exposes data from the **Model**. The key feature here is **data binding**, which allows automatic synchronization between the View and ViewModel. Unlike MVP, the ViewModel does not directly update the View. This pattern works well with frameworks like **WPF**, **Xamarin**, and **Angular**, where data binding is supported.



When to Use Which:

Use MVP (Model-View-Presenter) when:

- You're building apps without strong data binding support (like Android or Windows Forms).
- Testing is a priority, since logic is in the Presenter and not tied to UI.
- You want full control over how data is passed and displayed.

Why? Because MVP separates UI and logic clearly, making it easier to manage and test.

Use MVVM (Model-View-ViewModel) when:

- You're using frameworks that support **data binding** (like WPF, Xamarin, Angular).
- You want less code for UI updates (binding handles it).
- You aim for a clean separation between UI and business logic.

Why? MVVM reduces boilerplate and is ideal for dynamic UIs where the ViewModel can automatically update the View.